

19 Step-by-step order	Topics	100+ Problems Most important listed	JavaScript All code in JS	Zero to Hero Starting from scratch
---------------------------------	---------------	--	----------------------------------	---

This guide covers every topic you need to go from zero programming knowledge to solving DSA problems confidently in JavaScript interviews. Follow the order exactly — each topic builds on the previous one.

Table of Contents

Step 0 — Before You Start — JavaScript Prerequisites

Step 1 — Time Complexity — Big O Notation

Step 2 — Recursion Basics

Step 3 — Arrays — The Most Important Topic

Step 4 — Strings

Step 5 — HashMap and HashSet

Step 6 — Sorting Algorithms

Step 7 — Binary Search

Step 8 — Linked List

Step 9 — Stack

Step 10 — Queue and Deque

Step 11 — Trees — Binary Tree and BST

Step 12 — Heap and Priority Queue

Step 13 — Graphs

Step 14 — Trie

Step 15 — Backtracking

Step 16 — Dynamic Programming

Step 17 — Greedy Algorithms

Step 18 — Bit Manipulation

Step A — Daily Study Routine

Step B — Problem-Solving Framework

Step C — Must-Solve Problem List

Step 0 — JavaScript Prerequisites

Before touching a single DSA problem, you must be completely comfortable with these JavaScript fundamentals. If any of these feel shaky, spend a few days reviewing them first.

Array Methods You Must Know

push() / **pop()** — Add/remove from end — $O(1)$
shift() / **unshift()** — Add/remove from start — $O(n)$, slow!
slice(start, end) — Returns new array, does NOT mutate original
splice(index, count) — Mutates original — removes/inserts elements
concat() — Joins arrays, returns new array
sort(compareFn) — Default sort is alphabetical — always pass compareFn for numbers
reverse() — Mutates original, reverses in-place
includes() / **indexOf()** — Linear search $O(n)$
find() / **findIndex()** — Returns first match or -1
filter() — Returns new array of elements passing the test
map() — Returns new array of transformed elements
reduce() — Reduces array to single value — master this
some() / **every()** — Returns boolean — any/all elements pass
flat() / **flatMap()** — Flattens nested arrays

String Methods You Must Know

split(delimiter) — String to array — `split("")` splits every character
join(delimiter) — Array to string
substring(start, end) — Extracts part of string
slice(start, end) — Like substring but accepts negative indices
replace() / **replaceAll()** — Replace occurrences
trim() — Remove whitespace from both ends
includes() / **startsWith()** / **endsWith()** — Checks for substring
repeat(n) — Repeats string n times
charAt(i) / **charCodeAt(i)** — Get character or ASCII code at index
toUpperCase() / **toLowerCase()** — Case conversion

ES6+ Features Critical for DSA

These make your solutions shorter and cleaner:

Destructuring — `const [a, b] = arr; const {x, y} = obj;`
Spread operator — `[...arr1, ...arr2]` — copy or merge arrays
Rest operator — `function f(...args)` — collect into array

Arrow functions — $(a, b) \Rightarrow a + b$ — shorter syntax

Template literals — ``Hello ${name}`` — embed expressions in strings

let / const — Always use `const` by default, `let` only when reassigning

Optional chaining — `obj?.property` — avoid null reference errors

Nullish coalescing — `a ?? b` — use `b` only if `a` is null/undefined

Important: Sorting numbers correctly in JavaScript

```
// WRONG – default sort is alphabetical
[10, 2, 21].sort()           // [10, 2, 21] – wrong!

// CORRECT – always pass compareFn for numbers
[10, 2, 21].sort((a, b) => a - b) // [2, 10, 21] ascending
[10, 2, 21].sort((a, b) => b - a) // [21, 10, 2] descending
```

Tip: If you can solve basic problems with arrays, strings, and objects comfortably in JS, you are ready to move to Step 1.

Step 1 — Time Complexity (Big O Notation)

This is the single most important concept in DSA. You must understand it before solving any problem. Big O describes how your algorithm's runtime scales as the input size grows.

Complexity	Name	Example
$O(1)$	Constant	Array access <code>arr[i]</code> , HashMap <code>get/set</code>
$O(\log n)$	Logarithmic	Binary search, balanced BST operations
$O(n)$	Linear	Single loop through array, linear search
$O(n \log n)$	Linearithmic	Merge sort, Quick sort (average)
$O(n^2)$	Quadratic	Nested loops, Bubble/Selection sort
$O(2^n)$	Exponential	Recursive subsets, Fibonacci (naive)
$O(n!)$	Factorial	Permutations, Traveling salesman (brute)

How to Calculate Complexity

Rule 1 — Drop constants. $O(2n) = O(n)$, $O(100) = O(1)$

Rule 2 — Drop lower-order terms. $O(n^2 + n) = O(n^2)$

Rule 3 — Single loop = $O(n)$. One pass through n elements.

Rule 4 — Nested loops = multiply. Loop inside loop = $O(n) \times O(n) = O(n^2)$.

Rule 5 — Halving = $O(\log n)$. Each step cuts the problem in half.

Complexity examples in JavaScript

```
// O(1) – constant time
function getFirst(arr) {
  return arr[0];
}

// O(n) – linear time
function findMax(arr) {
  let max = arr[0];
  for (let i = 1; i < arr.length; i++) { // one loop = O(n)
    if (arr[i] > max) max = arr[i];
  }
  return max;
}

// O(n^2) – quadratic time
function hasDuplicate(arr) {
  for (let i = 0; i < arr.length; i++) { // outer loop O(n)
    for (let j = i + 1; j < arr.length; j++) { // inner loop O(n)
      if (arr[i] === arr[j]) return true; // = O(n^2) total
    }
  }
  return false;
}

// O(log n) – binary search halves input each step
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (arr[mid] === target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
}
```

Space Complexity

Space complexity measures extra memory your algorithm uses (not counting the input itself).

$O(1)$ space: only a few variables, no extra arrays

$O(n)$ space: creating a new array of size n , or recursion depth n

$O(n^2)$ space: 2D matrix of size $n \times n$

Watch out: Interviewers always ask 'what is the time and space complexity?' — never forget to mention both after your solution.

Tip: $O(n^2)$ is usually the brute force. Think: can I do this in $O(n)$ or $O(n \log n)$? That thought process alone will push you toward optimal solutions.

Step 2 — Recursion

Recursion is when a function calls itself to solve a smaller version of the same problem. It is the foundation of tree traversal, backtracking, and dynamic programming.

The Three Laws of Recursion

- 1. Base case:** A condition that stops the recursion. Without this, you get infinite loop and stack overflow.
- 2. Recursive case:** The function calls itself with a SMALLER input.
- 3. Progress:** Each call must move toward the base case.

Factorial — simplest recursion example

```
function factorial(n) {  
    if (n <= 1) return 1;          // BASE CASE – stop here  
    return n * factorial(n - 1);  // RECURSIVE CASE – smaller input  
}  
  
// factorial(4)  
// = 4 * factorial(3)  
// = 4 * 3 * factorial(2)  
// = 4 * 3 * 2 * factorial(1)  
// = 4 * 3 * 2 * 1 = 24
```

Fibonacci — classic recursion (then we optimize)

```
// Naive recursion – O(2^n) time – too slow!  
function fib(n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}  
  
// With memoization – O(n) time (this is DP!)  
function fibMemo(n, memo = {}) {  
    if (n in memo) return memo[n];  
    if (n <= 1) return n;  
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);  
    return memo[n];  
}
```

Reverse a string recursively

```
function reverseStr(s) {  
  if (s.length <= 1) return s;           // base case  
  return reverseStr(s.slice(1)) + s[0];  // recursive case  
}  
  
// reverseStr("hello")  
// = reverseStr("ello") + "h"  
// = reverseStr("llo") + "e" + "h"  
// ... = "olleh"
```

Call Stack Visualized

Each recursive call is pushed onto the call stack. When the base case is hit, calls unwind (pop off) in reverse order. JavaScript's default call stack limit is around 10,000 — deep recursion can cause 'Maximum call stack size exceeded'.

Problems to Practice

- Factorial of n **[Easy]**
- Sum of array **[Easy]**
- Power(x, n) **[Easy]**
- Reverse a string **[Easy]**
- Palindrome check **[Easy]**
- Fibonacci (memoized) **[Medium]**
- Tower of Hanoi **[Medium]**
- Flatten nested array **[Medium]**
- Generate all subsets **[Medium]**
- Permutations of string **[Hard]**

Tip: Draw the recursion tree on paper before coding. If you can draw it, you can code it.

Watch out: Always identify the base case FIRST. Writing it last causes bugs and infinite loops.

Step 3 — Arrays (Most Important Topic)

Arrays alone account for close to 100 interview questions. Master these patterns and you will be able to solve the majority of array problems by recognizing which technique applies.

Pattern 1 — Two Pointers

Use two pointers (left and right, or slow and fast) moving through the array. Eliminates the need for a nested loop, reducing $O(n^2)$ to $O(n)$.

Two pointers — check if array has pair summing to target

```
function hasPairWithSum(arr, target) {  
  let left = 0;  
  let right = arr.length - 1;  
  
  // Array must be sorted first for this approach  
  arr.sort((a, b) => a - b);  
  
  while (left < right) {  
    const sum = arr[left] + arr[right];  
    if (sum === target) return true;  
    else if (sum < target) left++;    // need bigger sum  
    else right--;                    // need smaller sum  
  }  
  return false;  
}
```

Pattern 2 — Sliding Window

Maintain a 'window' of elements. Expand the right edge, shrink the left edge based on a condition. Converts $O(n^2)$ nested approach to $O(n)$.

Sliding window — max sum of subarray of size k

```
function maxSumSubarray(arr, k) {  
    let windowSum = 0;  
    let maxSum = 0;  
  
    // Build first window  
    for (let i = 0; i < k; i++) {  
        windowSum += arr[i];  
    }  
    maxSum = windowSum;  
  
    // Slide the window: add right element, remove left element  
    for (let i = k; i < arr.length; i++) {  
        windowSum += arr[i] - arr[i - k]; // O(1) per step  
        maxSum = Math.max(maxSum, windowSum);  
    }  
    return maxSum;  
}
```

Pattern 3 — Prefix Sum

Precompute cumulative sums so any range sum query becomes $O(1)$. Build the prefix array once in $O(n)$, then answer queries in $O(1)$.

Prefix sum — range sum query

```
function buildPrefix(arr) {  
    const prefix = [0];  
    for (let i = 0; i < arr.length; i++) {  
        prefix.push(prefix[i] + arr[i]);  
    }  
    return prefix;  
}  
  
// Sum from index l to r (inclusive) = O(1)  
function rangeSum(prefix, l, r) {  
    return prefix[r + 1] - prefix[l];  
}  
  
// Example: arr = [1, 2, 3, 4, 5]  
// prefix = [0, 1, 3, 6, 10, 15]  
// rangeSum(prefix, 1, 3) = prefix[4] - prefix[1] = 10 - 1 = 9 (2+3+4)
```

Pattern 4 — Kadane's Algorithm (Max Subarray Sum)

Track the maximum sum ending at each position. If the current sum drops below 0, reset it — a negative prefix only hurts us.

Kadane's algorithm — $O(n)$ time, $O(1)$ space

```
function maxSubarraySum(arr) {  
  let currentSum = arr[0];  
  let maxSum = arr[0];  
  
  for (let i = 1; i < arr.length; i++) {  
    // Either extend the previous subarray or start fresh  
    currentSum = Math.max(arr[i], currentSum + arr[i]);  
    maxSum = Math.max(maxSum, currentSum);  
  }  
  
  return maxSum;  
}  
  
// arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]  
// Answer: 6 (subarray [4, -1, 2, 1])
```

Essential Array Problems by Priority

Problem	Technique	Difficulty	Priority
Two Sum	HashMap	Easy	Must know
Best Time to Buy/Sell	Sliding window	Easy	Must know
Maximum subarray	Kadane's	Medium	Must know
Product except self	Prefix sum	Medium	Must know
Container with most water	Two pointers	Medium	Must know
3Sum	Two pointers	Medium	Very high
Merge intervals	Sorting	Medium	Very high
Search rotated array	Binary search	Medium	Very high
Move zeroes	Two pointers	Easy	High
Sort 0s 1s 2s	Three pointers	Medium	High
Missing number	XOR / Gauss	Easy	High
Find duplicate number	Floyd cycle	Medium	High
Rotate array	Triple reverse	Medium	Medium
Subarray sum equals K	Prefix + map	Medium	Medium
Spiral matrix	Simulation	Medium	Medium

Step 4 — Strings

Strings in JavaScript are immutable — you cannot change a character in place. Always split into an array if you need to mutate. Many string problems are array problems in disguise.

Key Insight: Strings Are Arrays of Characters

String fundamentals in JavaScript

```
const s = "hello";

// Iterate over characters
for (const ch of s) console.log(ch);

// Get char at index
s[0];           // 'h'
s.charAt(0);    // 'h'

// Get ASCII code (vital for problems)
s.charCodeAt(0); // 104
String.fromCharCode(104); // 'h'

// Check if lowercase letter: charCode between 97 and 122
// Check if uppercase:      charCode between 65 and 90
// Check if digit:          charCode between 48 and 57

// Split to array to manipulate
const arr = s.split(''); // ['h','e','l','l','o']
arr.reverse();
arr.join('');             // 'olleh'
```

Pattern: Frequency Counter with Map

Anagram check — $O(n)$ time

```
function isAnagram(s, t) {
  if (s.length !== t.length) return false;

  const freq = new Map();
  for (const ch of s) freq.set(ch, (freq.get(ch) || 0) + 1);
  for (const ch of t) {
    if (!freq.has(ch) || freq.get(ch) === 0) return false;
    freq.set(ch, freq.get(ch) - 1);
  }
  return true;
}
```

Pattern: Sliding Window on String

Longest substring without repeating characters

```
function lengthOfLongestSubstring(s) {  
  const seen = new Map(); // char -> last seen index  
  let left = 0;  
  let maxLen = 0;  
  
  for (let right = 0; right < s.length; right++) {  
    const ch = s[right];  
    // If char seen and it's inside our window, shrink from left  
    if (seen.has(ch) && seen.get(ch) >= left) {  
      left = seen.get(ch) + 1;  
    }  
    seen.set(ch, right);  
    maxLen = Math.max(maxLen, right - left + 1);  
  }  
  return maxLen;  
}
```

Must-Know String Problems

Problem	Technique	Difficulty	Priority
Valid palindrome	Two pointers	Easy	Must know
Reverse words in string	Split + reverse	Medium	Must know
Longest substring no repeat	Sliding window	Medium	Must know
Valid anagram	Frequency map	Easy	Must know
Group anagrams	HashMap	Medium	Very high
Longest palindromic substring	Expand around center	Medium	Very high
String to integer (atoi)	Parsing	Medium	High
Count and say	Simulation	Medium	Medium
Word break	DP + trie	Medium	Medium
Minimum window substring	Sliding window	Hard	High

Tip: When a string problem involves character frequency, always reach for a Map or a frequency array of size 26 (for lowercase letters).

Step 5 — HashMap and HashSet

HashMap (Map in JavaScript) and HashSet (Set in JavaScript) give you $O(1)$ average time for insert, delete, and lookup. They are the most powerful tools for reducing time complexity.

Map — complete API

```
const map = new Map();

// Set a key-value pair
map.set('name', 'Alice');
map.set(1, 'one');           // keys can be any type

// Get a value
map.get('name');             // 'Alice'
map.get('missing');          // undefined

// Check existence
map.has('name');             // true

// Delete
map.delete('name');

// Size
map.size;

// Iterate
for (const [key, val] of map) {
  console.log(key, val);
}

// Common pattern: count with default 0
const freq = new Map();
for (const x of arr) {
  freq.set(x, (freq.get(x) || 0) + 1);
}
```

Set — complete API

```
const set = new Set();

set.add(5);
set.add(5);    // duplicates silently ignored
set.has(5);    // true
set.delete(5);
set.size;      // 0

// Create from array (removes duplicates)
const unique = new Set([1, 2, 2, 3, 3, 3]); // {1, 2, 3}
[...unique]   // back to array: [1, 2, 3]

// Set intersection
const a = new Set([1, 2, 3]);
const b = new Set([2, 3, 4]);
const intersection = new Set([...a].filter(x => b.has(x))); // {2, 3}

// Fastest way to deduplicate an array
const deduped = [...new Set(arr)];
```

Two Sum — the classic HashMap problem

```
function twoSum(nums, target) {
  const seen = new Map(); // value -> index

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (seen.has(complement)) {
      return [seen.get(complement), i];
    }
    seen.set(nums[i], i);
  }
  return [];
}

// O(n) time, O(n) space – vs O(n^2) brute force
```

Must-Know HashMap/HashSet Problems

Problem	Technique	Difficulty	Priority
Two Sum	HashMap	Easy	Must know
Contains duplicate	HashSet	Easy	Must know
Valid anagram	HashMap	Easy	Must know
Group anagrams	HashMap	Medium	Very high
Top K frequent elements	HashMap+Heap	Medium	Very high
Longest consecutive sequence	HashSet	Medium	Very high
First unique character	HashMap	Easy	High
Subarray sum equals K	HashMap	Medium	High
4Sum II	HashMap	Medium	Medium
Word pattern	HashMap	Easy	Medium

Step 6 — Sorting Algorithms

You do not need to memorize every sorting algorithm. Understand WHY each one works, its complexity, and whether it is stable and in-place. In interviews, you will mainly use the built-in sort and know when to apply merge sort logic.

Algorithm	Best	Average	Worst	Space	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes	No

Merge Sort — the most important to understand

```
function mergeSort(arr) {  
  if (arr.length <= 1) return arr; // base case  
  
  const mid = Math.floor(arr.length / 2);  
  const left = mergeSort(arr.slice(0, mid));  
  const right = mergeSort(arr.slice(mid));  
  
  return merge(left, right);  
}  
  
function merge(left, right) {  
  const result = [];  
  let i = 0, j = 0;  
  
  while (i < left.length && j < right.length) {  
    if (left[i] <= right[j]) result.push(left[i++]);  
    else result.push(right[j++]);  
  }  
  
  // Append remaining elements  
  return result.concat(left.slice(i)).concat(right.slice(j));  
}  
  
// Time:  $O(n \log n)$  always | Space:  $O(n)$  for merge buffer
```

Tip: JavaScript's built-in `arr.sort((a,b) => a-b)` uses TimSort (hybrid of merge sort and insertion sort) — $O(n \log n)$. Use it in interviews unless asked to implement sorting yourself.

Watch out: Never use `arr.sort()` without a comparator for numbers. Default sort converts to strings: `[10, 9, 2].sort()` gives `[10, 2, 9]`.

Step 7 — Binary Search

Binary search finds a target in a SORTED collection in $O(\log n)$ by halving the search space each step. It is also used to search on an 'answer space' — when you can check if a value satisfies a condition.

Classic binary search — memorize this template

```
function binarySearch(arr, target) {
  let lo = 0;
  let hi = arr.length - 1;

  while (lo <= hi) {           // NOTE: <= not <
    const mid = Math.floor((lo + hi) / 2); // avoids overflow vs (lo+hi)/2
    if (arr[mid] === target) return mid;    // found
    else if (arr[mid] < target) lo = mid + 1; // target in right half
    else hi = mid - 1;                      // target in left half
  }
  return -1; // not found
}
```

Search in rotated sorted array — important variant

```
function searchRotated(arr, target) {
  let lo = 0, hi = arr.length - 1;

  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (arr[mid] === target) return mid;

    // Left half is sorted
    if (arr[lo] <= arr[mid]) {
      if (arr[lo] <= target && target < arr[mid]) hi = mid - 1;
      else lo = mid + 1;
    }

    // Right half is sorted
    else {
      if (arr[mid] < target && target <= arr[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}
```

Binary Search on Answer Space

This is the advanced pattern. When asked 'find the minimum/maximum value satisfying some condition', binary search on the possible answer values, not on the array.

Binary search on answer — find $\text{sqrt}(n)$ without `Math.sqrt`

```
function mySqrt(n) {  
  if (n < 2) return n;  
  let lo = 1, hi = Math.floor(n / 2);  
  
  while (lo <= hi) {  
    const mid = Math.floor((lo + hi) / 2);  
    if (mid * mid === n) return mid;  
    else if (mid * mid < n) lo = mid + 1;  
    else hi = mid - 1;  
  }  
  return hi; // floor of sqrt  
}
```

Must-Know Binary Search Problems

Problem	Technique	Difficulty	Priority
Binary search (classic)	Template	Easy	Must know
Search insert position	Binary search	Easy	Must know
Search in rotated array	Modified BS	Medium	Must know
Find peak element	Modified BS	Medium	Very high
Find minimum in rotated	Modified BS	Medium	Very high
Koko eating bananas	BS on answer	Medium	High
Capacity to ship packages	BS on answer	Medium	High
Median of two sorted arrays	Binary search	Hard	Medium
Find first and last position	Lower/upper bound	Medium	High

Tip: If you see 'sorted array' + 'find target efficiently', binary search. If you see 'minimum/maximum satisfying a condition', binary search on the answer space.

Step 8 — Linked List

A linked list is a chain of nodes where each node holds a value and a pointer to the next node. Unlike arrays, there is no index — you must traverse from the head. No random access.

Linked list node class in JavaScript

```
class ListNode {
  constructor(val, next = null) {
    this.val = val;
    this.next = next;
  }
}

// Build: 1 -> 2 -> 3 -> null
const head = new ListNode(1, new ListNode(2, new ListNode(3)));

// Traverse
let curr = head;
while (curr !== null) {
  console.log(curr.val);
  curr = curr.next;
}
```

Reverse a linked list — must know

```
function reverseList(head) {
  let prev = null;
  let curr = head;

  while (curr !== null) {
    const next = curr.next; // save next
    curr.next = prev;       // reverse pointer
    prev = curr;            // move prev forward
    curr = next;            // move curr forward
  }

  return prev; // prev is now the new head
}
```

Detect cycle — Floyd's tortoise and hare

```
function hasCycle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {  
    slow = slow.next;      // moves 1 step  
    fast = fast.next.next;  // moves 2 steps  
    if (slow === fast) return true; // they meet = cycle  
  }  
  return false;  
}
```

Find middle of linked list — two pointers

```
function findMiddle(head) {  
  let slow = head;  
  let fast = head;  
  
  while (fast !== null && fast.next !== null) {  
    slow = slow.next;  
    fast = fast.next.next;  
  }  
  return slow; // slow is at the middle when fast reaches end  
}
```

Must-Know Linked List Problems

Problem	Technique	Difficulty	Priority
Reverse linked list	Iterative/recursive	Easy	Must know
Find middle node	Fast/slow pointers	Easy	Must know
Linked list cycle	Floyd's algorithm	Easy	Must know
Merge two sorted lists	Two pointers	Easy	Must know
Remove nth from end	Two pointers	Medium	Very high
Palindrome linked list	Reverse half	Easy	High
Reorder list	Find mid + reverse	Medium	High
Add two numbers	Simulation	Medium	High
Copy list with rand ptr	HashMap	Medium	Medium
LRU Cache	HashMap + DLL	Medium	Medium

Tip: Draw the pointer operations on paper. Linked list bugs are almost always pointer order errors — save next before overwriting.

Step 9 — Stack

A stack is a LIFO (Last In, First Out) data structure. In JavaScript, use an array as a stack: `push()` to add, `pop()` to remove from the top. Stacks are perfect for 'undo' operations, matching brackets, and tracking the nearest greater/smaller element.

Stack operations in JavaScript

```
const stack = [];  
  
stack.push(10);    // [10]  
stack.push(20);    // [10, 20]  
stack.push(30);    // [10, 20, 30]  
  
stack.pop();        // 30 removed – [10, 20]  
stack[stack.length - 1]; // peek top = 20  
stack.length;       // 2
```

Valid parentheses — classic stack problem

```
function isValid(s) {  
    const stack = [];  
    const pairs = { ')': '(', '}': '{', ']': '[' };  
  
    for (const ch of s) {  
        if ('({['.includes(ch)) {  
            stack.push(ch); // opening bracket: push  
        } else {  
            // Closing bracket: must match top of stack  
            if (stack.length === 0 || stack[stack.length-1] !== pairs[ch]) {  
                return false;  
            }  
            stack.pop();  
        }  
    }  
  
    return stack.length === 0; // stack must be empty at end  
}
```

Monotonic stack — Next Greater Element

```
function nextGreaterElement(arr) {  
  const result = new Array(arr.length).fill(-1);  
  const stack = []; // stores indices  
  
  for (let i = 0; i < arr.length; i++) {  
    // Pop all elements smaller than current  
    while (stack.length > 0 && arr[stack[stack.length-1]] < arr[i]) {  
      const idx = stack.pop();  
      result[idx] = arr[i]; // current element is the NGE for idx  
    }  
    stack.push(i);  
  }  
  return result;  
}  
  
// arr = [4, 5, 2, 10, 8]  
// NGE = [5, 10, 10, -1, -1]
```

Must-Know Stack Problems

Problem	Technique	Difficulty	Priority
Valid parentheses	Stack	Easy	Must know
Min stack	Two stacks	Medium	Must know
Next greater element	Monotonic stack	Medium	Very high
Daily temperatures	Monotonic stack	Medium	Very high
Largest rectangle histogram	Monotonic stack	Hard	High
Evaluate reverse polish	Stack	Medium	High
Decode string	Stack	Medium	High
Asteroid collision	Stack	Medium	Medium

Tip: Monotonic stack is the advanced stack pattern. When you need 'next greater / next smaller element', think monotonic stack.

Step 10 — Queue and Dequeue

A queue is FIFO (First In, First Out). In JavaScript, use an array with `push()` to enqueue and `shift()` to dequeue — but `shift()` is $O(n)$. For performance-critical code, implement with a head pointer or use a deque. Queues power BFS.

Queue with head pointer — $O(1)$ enqueue and dequeue

```
class Queue {
  constructor() {
    this.data = {};
    this.head = 0;
    this.tail = 0;
  }

  enqueue(val) { this.data[this.tail++] = val; }
  dequeue()     { return this.data[this.head++]; }
  peek()       { return this.data[this.head]; }
  isEmpty()    { return this.head === this.tail; }
  size()       { return this.tail - this.head; }
}
```

BFS with queue — level-order traversal foundation

```
function bfs(root) {
  if (!root) return [];
  const result = [];
  const queue = [root]; // enqueue root

  while (queue.length > 0) {
    const levelSize = queue.length; // process one level at a time
    const level = [];

    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift(); // dequeue
      level.push(node.val);
      if (node.left) queue.push(node.left); // enqueue children
      if (node.right) queue.push(node.right);
    }

    result.push(level);
  }

  return result;
}
```

Sliding window maximum — Deque (monotonic queue)

```
function maxSlidingWindow(nums, k) {  
  const result = [];  
  const deque = []; // stores indices, front = max  
  
  for (let i = 0; i < nums.length; i++) {  
    // Remove indices outside the window  
    while (deque.length > 0 && deque[0] < i - k + 1) deque.shift();  
  
    // Remove indices with smaller values (they can never be max)  
    while (deque.length > 0 && nums[deque[deque.length-1]] < nums[i]) {  
      deque.pop();  
    }  
    deque.push(i);  
  
    if (i >= k - 1) result.push(nums[deque[0]]); // front is max  
  }  
  return result;  
}
```

Must-Know Queue Problems

Problem	Technique	Difficulty	Priority
Binary tree level order	BFS + queue	Medium	Must know
Sliding window maximum	Monotonic deque	Hard	Very high
Number of islands	BFS	Medium	Must know
Rotting oranges	Multi-source BFS	Medium	Very high
Walls and gates	Multi-source BFS	Medium	High
First negative in window	Deque	Medium	High

Step 11 — Trees

Trees are one of the biggest topics in DSA interviews. A binary tree is a tree where each node has at most two children (left and right). A BST (Binary Search Tree) adds the property: left child is smaller, right child is larger.

Tree node class and traversals

```
class TreeNode {
  constructor(val, left = null, right = null) {
    this.val = val;
    this.left = left;
    this.right = right;
  }
}

// Inorder (Left, Root, Right) – gives sorted order in BST
function inorder(root) {
  if (!root) return [];
  return [...inorder(root.left), root.val, ...inorder(root.right)];
}

// Preorder (Root, Left, Right)
function preorder(root) {
  if (!root) return [];
  return [root.val, ...preorder(root.left), ...preorder(root.right)];
}

// Postorder (Left, Right, Root)
function postorder(root) {
  if (!root) return [];
  return [...postorder(root.left), ...postorder(root.right), root.val];
}
```

Height and diameter of binary tree

```
// Height = longest path from root to leaf
function height(root) {
  if (!root) return 0;
  return 1 + Math.max(height(root.left), height(root.right));
}

// Diameter = longest path between any two nodes
function diameterOfBinaryTree(root) {
  let maxDiam = 0;

  function dfs(node) {
    if (!node) return 0;
    const left = dfs(node.left);
    const right = dfs(node.right);
    maxDiam = Math.max(maxDiam, left + right); // path through current node
    return 1 + Math.max(left, right); // return height
  }

  dfs(root);
  return maxDiam;
}
```

Lowest Common Ancestor (LCA) — very common interview problem

```
function lowestCommonAncestor(root, p, q) {
  if (!root || root === p || root === q) return root;
  const left = lowestCommonAncestor(root.left, p, q);
  const right = lowestCommonAncestor(root.right, p, q);
  // If both sides return non-null, current node is LCA
  if (left && right) return root;
  return left || right;
}
```

Must-Know Tree Problems

Problem	Technique	Difficulty	Priority
Inorder / Preorder / Postorder	DFS recursion	Easy	Must know
Level order traversal	BFS	Medium	Must know
Max depth of binary tree	DFS	Easy	Must know
Validate BST	DFS + bounds	Medium	Must know
Lowest common ancestor	DFS	Medium	Must know
Path sum	DFS	Easy	Very high
Max path sum	DFS post-order	Hard	Very high
Symmetric tree	DFS / BFS	Easy	High
Right side view	BFS / DFS	Medium	High
Construct from pre+inorder	Recursion	Medium	High
Kth smallest in BST	Inorder	Medium	High
Serialize / deserialize	BFS	Hard	Medium

Step 12 — Heap and Priority Queue

A heap is a complete binary tree satisfying the heap property: in a max-heap, every parent is greater than or equal to its children. A priority queue is an abstract data type backed by a heap. JavaScript has no built-in heap — you need to implement one or use a sorted structure.

Min-Heap implementation in JavaScript

```
class MinHeap {
  constructor() { this.heap = []; }

  push(val) {
    this.heap.push(val);
    this._bubbleUp(this.heap.length - 1);
  }

  pop() {
    const min = this.heap[0];
    const last = this.heap.pop();
    if (this.heap.length > 0) {
      this.heap[0] = last;
      this._sinkDown(0);
    }
    return min;
  }

  peek() { return this.heap[0]; }
  size() { return this.heap.length; }

  _bubbleUp(i) {
    while (i > 0) {
      const parent = Math.floor((i - 1) / 2);
      if (this.heap[parent] > this.heap[i]) {
        [this.heap[parent], this.heap[i]] = [this.heap[i], this.heap[parent]];
        i = parent;
      } else break;
    }
  }

  _sinkDown(i) {
    const n = this.heap.length;
    while (true) {
      let smallest = i;
      const l = 2*i+1, r = 2*i+2;
      if (l < n && this.heap[l] < this.heap[smallest]) smallest = l;
      if (r < n && this.heap[r] < this.heap[smallest]) smallest = r;
      if (smallest !== i) {
        [this.heap[i], this.heap[smallest]] = [this.heap[smallest], this.heap[i]];
        i = smallest;
      } else break;
    }
  }
}
```

```

    }
}

```

K largest elements — classic heap problem

```

function kLargest(arr, k) {
    const minHeap = new MinHeap();

    for (const num of arr) {
        minHeap.push(num);
        if (minHeap.size() > k) minHeap.pop(); // keep only k largest
    }

    return minHeap.heap; // contains k largest elements
}

// Why min-heap for k largest?
// We want to evict the SMALLEST when size exceeds k.
// Min-heap lets us find and remove the smallest in O(log k).

```

Must-Know Heap Problems

Problem	Technique	Difficulty	Priority
K largest elements	Min-heap size k	Easy	Must know
K smallest elements	Max-heap size k	Easy	Must know
Top K frequent elements	Heap + HashMap	Medium	Must know
Merge K sorted lists	Min-heap	Hard	Very high
Find median from stream	Two heaps	Hard	Very high
K closest points to origin	Max-heap	Medium	High
Task scheduler	Heap + greedy	Medium	High
Kth largest in stream	Min-heap	Easy	High

Step 13 — Graphs

A graph is a collection of nodes (vertices) connected by edges. It is the most general data structure — trees are graphs. Graphs can be directed or undirected, weighted or unweighted, cyclic or acyclic.

Graph representations in JavaScript

```
// Adjacency List – most common,  $O(V+E)$  space
const graph = {
  0: [1, 2],
  1: [0, 3],
  2: [0, 4],
  3: [1],
  4: [2],
};

// Build from edge list
function buildGraph(n, edges) {
  const graph = Array.from({length: n}, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u); // undirected
  }
  return graph;
}
```

DFS and BFS on graph

```
// DFS – recursive
function dfs(graph, node, visited = new Set()) {
  if (visited.has(node)) return;
  visited.add(node);
  console.log(node);
  for (const neighbor of graph[node]) {
    dfs(graph, neighbor, visited);
  }
}

// BFS – iterative with queue
function bfs(graph, start) {
  const visited = new Set([start]);
  const queue = [start];

  while (queue.length > 0) {
    const node = queue.shift();
    console.log(node);
    for (const neighbor of graph[node]) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }
}
```

Topological sort — for DAGs (directed acyclic graphs)

```
function topoSort(numNodes, prerequisites) {  
  const graph = Array.from({length: numNodes}, () => []);  
  const inDegree = new Array(numNodes).fill(0);  
  
  for (const [course, prereq] of prerequisites) {  
    graph[prereq].push(course);  
    inDegree[course]++;  
  }  
  
  const queue = [];  
  for (let i = 0; i < numNodes; i++) {  
    if (inDegree[i] === 0) queue.push(i); // start with no prereqs  
  }  
  
  const order = [];  
  while (queue.length > 0) {  
    const node = queue.shift();  
    order.push(node);  
    for (const next of graph[node]) {  
      if (--inDegree[next] === 0) queue.push(next);  
    }  
  }  
  
  return order.length === numNodes ? order : []; // [] if cycle exists  
}
```

Must-Know Graph Problems

Problem	Technique	Difficulty	Priority
Number of islands	BFS / DFS	Medium	Must know
Clone graph	BFS + HashMap	Medium	Must know
Course schedule	Topological sort	Medium	Must know
Pacific Atlantic waterflow	Multi-source DFS	Medium	Very high
Walls and gates	Multi-source BFS	Medium	Very high
Network delay time	Dijkstra	Medium	Very high
Cheapest flights K stops	Bellman-Ford	Medium	High
Word ladder	BFS	Hard	High
Redundant connection	Union-Find	Medium	High
Graph valid tree	Union-Find / DFS	Medium	Medium

Step 14 — Trie (Prefix Tree)

A Trie is a tree where each node represents a character. It enables $O(L)$ insert and search where L is the word length — much faster than $O(n*L)$ with a list of strings. Use for autocomplete, spell checkers, and prefix matching.

Trie implementation in JavaScript

```
class TrieNode {
  constructor() {
    this.children = {}; // char -> TrieNode
    this.isEnd = false;
  }
}

class Trie {
  constructor() { this.root = new TrieNode(); }

  insert(word) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) {
        node.children[ch] = new TrieNode();
      }
      node = node.children[ch];
    }
    node.isEnd = true;
  }

  search(word) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) return false;
      node = node.children[ch];
    }
    return node.isEnd; // must end at a word boundary
  }

  startsWith(prefix) {
    let node = this.root;
    for (const ch of prefix) {
      if (!node.children[ch]) return false;
      node = node.children[ch];
    }
    return true; // don't need isEnd for prefix check
  }
}
```

Must-Know Trie Problems

Problem	Technique	Difficulty	Priority
Implement Trie	Trie basics	Medium	Must know
Word search II	Trie + backtrack	Hard	Very high
Design add/search words	Trie + wildcard	Medium	Very high
Replace words	Trie	Medium	High
Longest common prefix	Trie / sort	Easy	High

Step 15 — Backtracking

Backtracking explores all possible solutions by building candidates incrementally and abandoning (backtracking) when a candidate cannot lead to a valid solution. It is brute force with pruning.

The Universal Backtracking Template

Backtracking template — apply to all backtracking problems

```
function backtrack(path, choices) {  
    // Base case: path is complete  
    if (isComplete(path)) {  
        result.push([...path]); // save a copy  
        return;  
    }  
  
    for (const choice of choices) {  
        if (!isValid(path, choice)) continue; // pruning  
        path.push(choice); // make choice  
        backtrack(path, nextChoices(choice)); // explore  
        path.pop(); // undo choice (backtrack)  
    }  
}
```

Permutations — generate all arrangements

```
function permute(nums) {  
    const result = [];  
  
    function backtrack(path, remaining) {  
        if (remaining.length === 0) {  
            result.push([...path]);  
            return;  
        }  
        for (let i = 0; i < remaining.length; i++) {  
            path.push(remaining[i]);  
            backtrack(path, [...remaining.slice(0,i), ...remaining.slice(i+1)]);  
            path.pop();  
        }  
    }  
  
    backtrack([], nums);  
    return result;  
}
```

Subsets — generate all subsets (power set)

```
function subsets(nums) {  
  const result = [];  
  
  function backtrack(start, path) {  
    result.push([...path]); // every path is a valid subset  
  
    for (let i = start; i < nums.length; i++) {  
      path.push(nums[i]);  
      backtrack(i + 1, path); // i+1 to avoid reusing elements  
      path.pop();  
    }  
  }  
  
  backtrack(0, []);  
  return result;  
}
```

Must-Know Backtracking Problems

Problem	Technique	Difficulty	Priority
Subsets	Backtracking	Medium	Must know
Permutations	Backtracking	Medium	Must know
Combination sum	Backtracking	Medium	Must know
Word search	DFS + backtrack	Medium	Very high
Palindrome partitioning	Backtracking	Medium	Very high
N-Queens	Backtracking	Hard	High
Sudoku solver	Backtracking	Hard	Medium
Letter combinations phone	Backtracking	Medium	High

Tip: The key insight is always: make a choice, recurse, undo the choice. Never forget the undo step — it's what separates backtracking from brute force.

Step 16 — Dynamic Programming

Dynamic Programming (DP) solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation. It is the hardest topic — do not start here. You need recursion, arrays, and trees first.

The 4 Steps to Solving Any DP Problem

- 1. Identify if DP applies:** If the problem asks for 'maximum', 'minimum', 'count of ways', 'can it be done', and has overlapping subproblems — it's likely DP.
- 2. Define the state:** What does $dp[i]$ represent? Make it precise. Example: $dp[i]$ = maximum sum of subarray ending at index i .
- 3. Write the recurrence relation:** How does $dp[i]$ depend on previous values? Example: $dp[i] = \max(dp[i-1] + arr[i], arr[i])$.
- 4. Base case + order of computation:** What is $dp[0]$? $dp[1]$? Compute from smallest to largest subproblem.

The Learning Progression

Start with naive recursion → add memoization → convert to tabulation → optimize space.

House Robber — classic DP problem (all 4 stages)

```
const nums = [2, 7, 9, 3, 1];

// Stage 1: Naive recursion –  $O(2^n)$  too slow
function robNaive(i) {
  if (i < 0) return 0;
  return Math.max(robNaive(i-1), robNaive(i-2) + nums[i]);
}

// Stage 2: Memoization –  $O(n)$  time,  $O(n)$  space
function robMemo(i, memo = {}) {
  if (i < 0) return 0;
  if (i in memo) return memo[i];
  memo[i] = Math.max(robMemo(i-1, memo), robMemo(i-2, memo) + nums[i]);
  return memo[i];
}

// Stage 3: Tabulation (bottom-up DP) –  $O(n)$  time,  $O(n)$  space
function robDP(nums) {
  if (!nums.length) return 0;
  const dp = new Array(nums.length).fill(0);
  dp[0] = nums[0];
  dp[1] = Math.max(nums[0], nums[1]);
  for (let i = 2; i < nums.length; i++) {
    dp[i] = Math.max(dp[i-1], dp[i-2] + nums[i]);
  }
  return dp[nums.length - 1];
}

// Stage 4: Space optimized –  $O(n)$  time,  $O(1)$  space
function robOptimal(nums) {
  let prev2 = 0, prev1 = 0;
  for (const n of nums) {
    const curr = Math.max(prev1, prev2 + n);
    prev2 = prev1;
    prev1 = curr;
  }
  return prev1;
}
```

Must-Know DP Problems by Category

Problem	Technique	Difficulty	Priority
Climbing stairs / Fibonacci	1D DP	Easy	Must know
House Robber	1D DP	Medium	Must know
Coin Change	Unbounded KS	Medium	Must know
Longest Common Subsequence	2D DP	Medium	Must know
Longest Increasing Subsequence	1D DP / BSearch	Medium	Must know
0/1 Knapsack	2D DP	Medium	Very high
Word Break	1D DP	Medium	Very high
Unique paths	2D DP / Math	Medium	High
Jump Game	Greedy / DP	Medium	High
Partition equal subset sum	0/1 Knapsack	Medium	High
Edit distance	2D DP	Hard	High
Burst balloons	Interval DP	Hard	Medium

Step 17 — Greedy Algorithms

A greedy algorithm makes the locally optimal choice at each step, hoping to find a global optimum. Greedy works when the problem has the 'greedy choice property' — a local optimum leads to a global optimum. Proving this is often the hard part.

Jump Game — classic greedy

```
function canJump(nums) {  
  let maxReach = 0; // furthest index we can reach  
  
  for (let i = 0; i < nums.length; i++) {  
    if (i > maxReach) return false; // can't reach this index  
    maxReach = Math.max(maxReach, i + nums[i]);  
    if (maxReach >= nums.length - 1) return true;  
  }  
  return true;  
}
```

Merge intervals — greedy sorting

```
function merge(intervals) {  
  intervals.sort((a, b) => a[0] - b[0]); // sort by start time  
  const result = [intervals[0]];  
  
  for (let i = 1; i < intervals.length; i++) {  
    const last = result[result.length - 1];  
    const curr = intervals[i];  
  
    if (curr[0] <= last[1]) {  
      // Overlap – extend the last interval  
      last[1] = Math.max(last[1], curr[1]);  
    } else {  
      result.push(curr);  
    }  
  }  
  return result;  
}
```

Must-Know Greedy Problems

Problem	Technique	Difficulty	Priority
Jump Game	Greedy reach	Medium	Must know
Merge intervals	Sort + greedy	Medium	Must know
Gas station	Circular greedy	Medium	Very high
Task scheduler	Greedy + heap	Medium	Very high
Meeting rooms II	Sort + heap	Medium	High
Non-overlapping intervals	Greedy	Medium	High
Partition labels	Greedy	Medium	High
Candy	Two-pass greedy	Hard	Medium

Step 18 — Bit Manipulation

Bit manipulation lets you operate directly on binary representations of numbers. Useful for problems involving sets, power-of-two checks, finding unique elements, and space optimization.

Bit operators reference

```
// AND: both bits must be 1
5 & 3    // 101 & 011 = 001 = 1

// OR: either bit is 1
5 | 3    // 101 | 011 = 111 = 7

// XOR: bits differ (0 XOR 0 = 0, 1 XOR 1 = 0, 0 XOR 1 = 1)
5 ^ 3    // 101 ^ 011 = 110 = 6
n ^ n    // always 0 (key property!)
n ^ 0    // always n (key property!)

// NOT: flip all bits
~5       // -(5+1) = -6 in JavaScript

// Left shift: multiply by 2
5 << 1   // 10 = 10 (5 * 2)
1 << 3   // 8      (2^3)

// Right shift: divide by 2
20 >> 1  // 10     (20 / 2)

// Check if n is power of 2
(n & (n - 1)) === 0 && n > 0

// Get i-th bit
(n >> i) & 1

// Set i-th bit
n | (1 << i)

// Clear i-th bit
n & ~(1 << i)
```

Single Number — classic XOR problem

```
// Every number appears twice except one. Find the single one.
// XOR property: n ^ n = 0, n ^ 0 = n
function singleNumber(nums) {
    let result = 0;
    for (const n of nums) result ^= n;
    return result;
}

// nums = [4, 1, 2, 1, 2]
// 0 ^ 4 ^ 1 ^ 2 ^ 1 ^ 2 = 4 (pairs cancel out)
```

Must-Know Bit Manipulation Problems

Problem	Technique	Difficulty	Priority
Single number	XOR	Easy	Must know
Number of 1 bits	Bit count	Easy	Must know
Power of two	$n \& (n-1)$	Easy	Very high
Counting bits	DP + bits	Easy	Very high
Missing number	XOR	Easy	High
Reverse bits	Bit by bit	Easy	High
Sum of two integers	XOR + AND carry	Medium	Medium

Appendix A — Daily Study Routine

Consistency beats intensity. 2 hours every day beats 14 hours on Saturday. This routine is designed for someone targeting interviews in 3–6 months while working or studying.

The 3-4 Hour Daily Routine

Time	Activity	Details
0:00-0:45	Learn concept	Watch a video or read notes on today's topic. Don't skip — understanding beats memorization.
0:45-2:15	Solve problems	3-5 problems on today's topic. Don't look at solutions for at least 20 minutes per problem.
2:15-2:45	Review yesterday	Re-solve 2-3 problems from the previous day without looking at your solution.
2:45-3:30	Study solutions	Read optimal solutions for today's problems. Re-implement them from scratch — don't just copy.

Spaced Repetition Schedule

After solving a problem, review it on these days to lock it into long-term memory:

Review timing	When
First review	1 day after solving
Second review	7 days after solving
Third review	30 days after solving
Final check	Before interview

Phase-wise Timeline (3-month plan)

Month 1 — Steps 0-7: JavaScript prerequisites, Big O, Recursion, Arrays, Strings, HashMap, Sorting, Binary Search

Month 2 — Steps 8-13: Linked List, Stack, Queue, Trees, Heap, Graphs

Month 3 — Steps 14-18: Trie, Backtracking, Dynamic Programming, Greedy, Bit Manipulation + Mock interviews

Weekly Rhythm

Mon-Fri: Learn one topic per day + solve 3-5 problems

Saturday: Timed mock interview (LeetCode contest or random problems — 90 min)

Sunday: Review weak areas, re-solve hard problems, plan next week

Watch out: The biggest mistake is solving too many problems without understanding. Solving 200 problems you don't understand is worse than truly mastering 50.

Tip: Track every problem you solve in a spreadsheet: problem name, date solved, time taken, confidence (1-5). Review low-confidence problems weekly.

Appendix B — Problem-Solving Framework

When you see a problem in an interview, follow these exact steps. Never jump to coding immediately.

Step 1 — Read carefully (2 min)

- Read the problem statement completely.
- Note the input type and constraints (array, string, integer, etc.).
- Note the output type.
- Note edge cases: empty input, single element, all same, negative numbers, very large input.

Step 2 — Ask clarifying questions (1-2 min)

- 'Is the array sorted?' — could enable binary search.
- 'Can there be duplicates?' — affects counting problems.
- 'What are the constraints on n?' — hints at expected time complexity.
- 'Can I modify the input array in-place?'

Step 3 — Think of examples (2 min)

- Work through 1-2 examples by hand.
- Check your examples against the expected output.
- If you cannot trace through an example, you don't understand the problem yet.

Step 4 — Identify the pattern (3 min)

- Which category does this fall into? (Two pointers? Sliding window? BFS? DP?)
- What data structure would help? (HashMap for lookup, Stack for LIFO, Heap for K-th element).
- Start with the brute force — say it out loud, then think about how to improve it.

Step 5 — State your approach before coding (1 min)

- Say exactly what algorithm you'll use.
- 'I'll use a sliding window. Left pointer starts at 0, right expands until the window is invalid...'
- Get interviewer buy-in before writing code.

Step 6 — Code cleanly (15-20 min)

- Use meaningful variable names (left/right not l/r for two pointers).
- Add a comment for non-obvious logic.
- Code in the order you explained your approach.
- If you get stuck, say so and talk through your thinking.

Step 7 — Test your code (3-5 min)

- Trace through your own example manually.
- Test the edge cases you noted in Step 1.
- Check for off-by-one errors ($<$ vs $\leq$, index 0 vs 1, etc.).

Step 8 — Analyze complexity

- Always state time complexity AND space complexity.
- 'This runs in $O(n)$ time and uses $O(n)$ space for the HashMap.'

Pattern Recognition Cheat Sheet

If you see...	Think...
Sorted array + find target	Binary search
Find pair/triplet summing to X	Two pointers or HashMap
Longest/shortest subarray satisfying X	Sliding window
Count subarrays with sum K	Prefix sum + HashMap
Max/min across subarrays	Kadane's or sliding window
Duplicate detection	HashSet or XOR
K-th largest/smallest	Heap (size k)
Level-by-level tree	BFS + queue
Path in tree (root to leaf)	DFS recursion
Matching brackets / undo operations	Stack
Next greater/smaller element	Monotonic stack
Number of ways / can we do X	Dynamic programming
Generate all possibilities	Backtracking
Shortest path, unweighted	BFS
Shortest path, weighted	Dijkstra
Ordering with dependencies	Topological sort
String prefix matching	Trie

Appendix C — Master Problem List

These are the problems to solve before any product company interview. Solve them in order. Mark each one as: Not started / Understood / Solved without help / Solved in optimal time.

Arrays — Sliding Window & Two Pointers

#	Problem	Difficulty	Status
1	Two Sum	Easy	[]
2	Best time to buy/sell stock	Easy	[]
3	Contains duplicate	Easy	[]
4	Move zeroes	Easy	[]
5	Maximum subarray (Kadane)	Medium	[]
6	3Sum	Medium	[]
7	Container with most water	Medium	[]
8	Product of array except self	Medium	[]
9	Maximum product subarray	Medium	[]
10	Merge intervals	Medium	[]

Arrays — Binary Search & Others

#	Problem	Difficulty	Status
1	Find minimum in rotated sorted array	Medium	[]
2	Search in rotated sorted array	Medium	[]
3	Find peak element	Medium	[]
4	Missing number	Easy	[]
5	Find the duplicate	Medium	[]
6	Rotate array	Medium	[]
7	Subarray sum equals K	Medium	[]

Strings

#	Problem	Difficulty	Status
1	Valid palindrome	Easy	[]
2	Valid anagram	Easy	[]
3	Longest substring without repeating	Medium	[]
4	Reverse words in string	Medium	[]
5	Group anagrams	Medium	[]
6	Longest palindromic substring	Medium	[]
7	Minimum window substring	Hard	[]

Stack & Queue

#	Problem	Difficulty	Status
1	Valid parentheses	Easy	[]
2	Min stack	Medium	[]
3	Next greater element	Medium	[]
4	Daily temperatures	Medium	[]
5	Largest rectangle in histogram	Hard	[]
6	Sliding window maximum	Hard	[]

Linked List

#	Problem	Difficulty	Status
1	Reverse linked list	Easy	[]
2	Linked list cycle	Easy	[]
3	Find middle of linked list	Easy	[]
4	Merge two sorted lists	Easy	[]
5	Remove nth node from end	Medium	[]
6	Reorder list	Medium	[]
7	LRU Cache	Medium	[]

Trees

#	Problem	Difficulty	Status
1	Inorder traversal	Easy	[]
2	Max depth	Easy	[]
3	Symmetric tree	Easy	[]
4	Path sum	Easy	[]
5	Level order traversal	Medium	[]
6	Validate BST	Medium	[]
7	Lowest common ancestor	Medium	[]
8	Max path sum	Hard	[]
9	Serialize/deserialize	Hard	[]

Graphs

#	Problem	Difficulty	Status
1	Number of islands	Medium	[]
2	Clone graph	Medium	[]
3	Course schedule	Medium	[]
4	Pacific Atlantic waterflow	Medium	[]
5	Number of connected components	Medium	[]
6	Word ladder	Hard	[]

Dynamic Programming

#	Problem	Difficulty	Status
1	Climbing stairs	Easy	[]
2	House robber	Medium	[]
3	Coin change	Medium	[]
4	Longest common subsequence	Medium	[]
5	Longest increasing subsequence	Medium	[]
6	Word break	Medium	[]
7	Unique paths	Medium	[]
8	Jump game	Medium	[]
9	Partition equal subset sum	Medium	[]
10	Edit distance	Hard	[]

Heap & Greedy & Backtracking

#	Problem	Difficulty	Status
1	K largest elements	Easy	[]
2	Top K frequent elements	Medium	[]
3	Find median from stream	Hard	[]
4	Subsets	Medium	[]
5	Permutations	Medium	[]
6	Combination sum	Medium	[]
7	Word search	Medium	[]
8	N-Queens	Hard	[]